

KindMind

A Mental Wellbeing Companion

CFGDegree — Software & Data Engineering
Group Project Report
June 2026

Github Repo:

<https://github.com/khanhbui37/CFG-KindMind-Project>

1. Introduction

1.1. Project Overview & Business Value

KindMind is a free, Flask-based mental well-being app addressing the gap in accessible emotional health tools. Users log moods, journal thoughts, and receive personalised motivation. It targets preventative mental health — individuals seeking consistent self-care routines, not crisis intervention. Core value: intuitive interface, privacy-focused, destigmatising emotional check-ins, and extensible into a SaaS product with premium features.

1.2. Problem Statement

In the fast-paced digital world, people constantly chase new phenomena without taking breaks, driven by fear of missing out. These relentless pursuits drain their energy, leaving them exhausted both physically and mentally. Compounded by environmental factors like weather changes that influence mood and well-being, they struggle to understand or express their emotions truly, lacking accessible tools and spaces to pause, reflect, and prioritise their emotional well-being.

1.3. Solutions Implemented

Users will be able to journal their emotions throughout the day by writing, search for a keyword that represents their emotions, and receive suggestions based on mood and context. It can also use Weather as an indicator or a factor affecting their moods.

1.4. Sprint Overview

Sprint 1: Choose project idea | Documentation | Create repo | Submit project assignment | README setup | SQL database & queries | Console UI framework

Sprint 2: Menu Login | Improve Database | Journal entry UI | Starting API | Unit tests

Sprint 3: Improve Login features | Journal entry UI | Database CRUD operations | Review functionalities | Remove unnecessary files | Implementing hashing | Integrate view/edit/delete with API | Final testing | Integration testing flow | Demo recording

Sprint 4: Finalise Project Report | Presentation | Project objectives & final result | Live Demo

1.5. Report Roadmap

Section 2 — Background: context on mental well-being tracking and the problem KindMind solves.

Section 3 — Specifications & Design: Requirements and Architectural Decisions.

Section 4 — Implementation & Execution: development process, team roles, tools, and challenges.

Section 5 — Testing & Evaluation: testing strategy, coverage, and known limitations

Section 6 — Conclusion: outcomes against the original aims.

2. Background / Product Backlog & User Stories

Using Agile methodology, the team defined the following Product Backlog goals: user account management (open registration, login, profile), mood logging (1–5 scale with optional journal entry), mood history & trend summaries, curated mental health resources via external API, secure MySQL storage, and web-based interface.

Domain rules: open registration (no verification barriers), private user accounts, mood logs record date/time/score/note, external content triggers based on mood score, persistent MySQL storage, passwords hashed with bcrypt (never plain text), browser-accessible with no installation required.

3. Specifications & Design

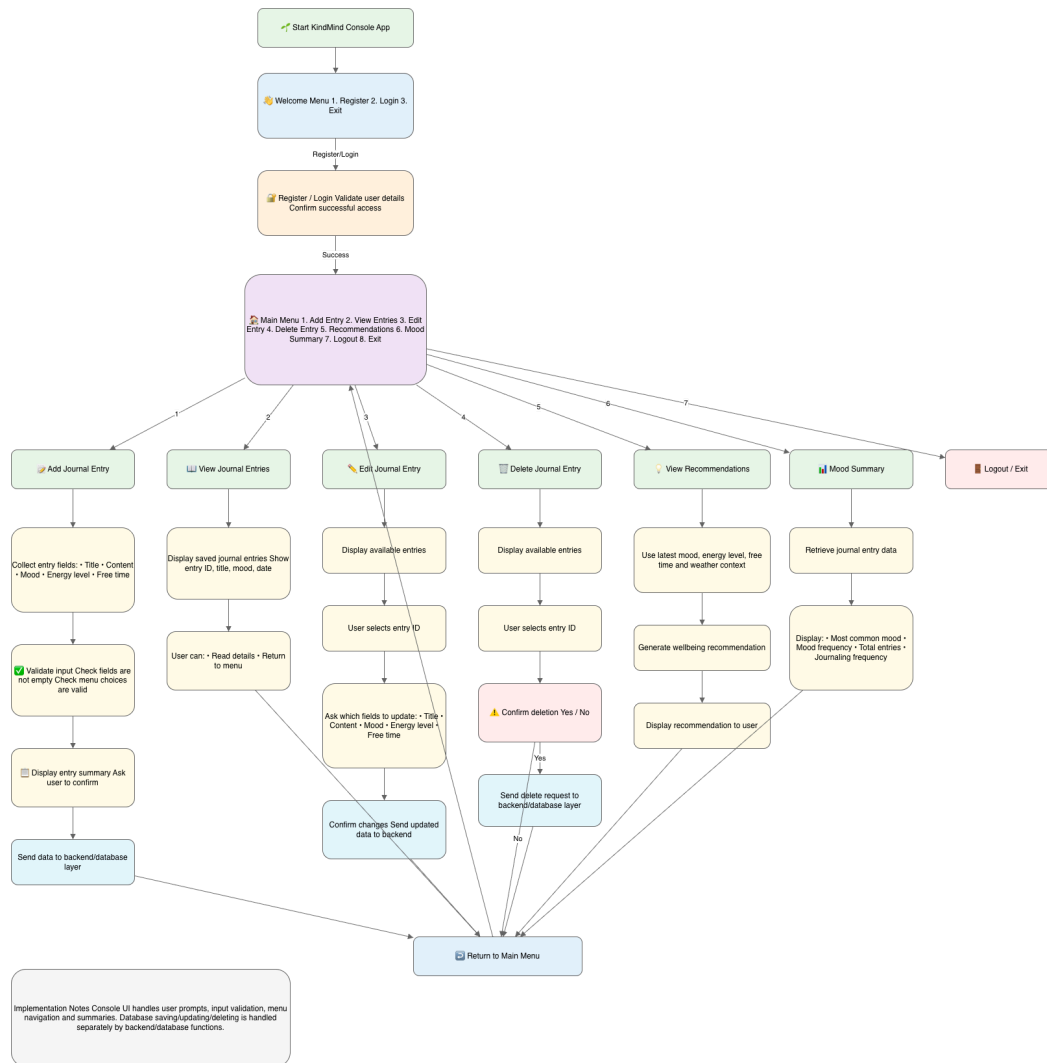
3.1 Non-Technical Requirements

The application must be accessible via a standard web browser with no installation required. All users can sign up freely, and the application must handle the process smoothly with no data loss. UI must be intuitive, with clear prompts, feedback messages, and error states.

3.2 Technical Requirements & System Specifications

Requirement	Detail
Language	Python 3.10+
Web Framework	Flask 3.x — routing, request handling, templating (Jinja2)
Database	MySQL 8.x , DBeaver
Authentication	Flask session management; bcrypt password hashing; open registration
CRUD Operations	Create, Read, Update, Delete for Users, MoodLogs, and JournalEntries
External API	OpenWeather API - mood-triggered motivational content
Specialist Topic	Search/sort algorithm — binary search on sorted mood log history by date
Unit Testing	Python unittest — covering auth, CRUD, search, and exception handling
Exception Handling	Try/except blocks throughout; custom 404 and 500 error pages
Version Control	Git with GitHub; feature branches per team member and sprint

3.5 User Flow Diagram



4. Implementation & Execution

4.1 Team Roles & Workload Distribution

Team Member	Role	Responsibilities
Khanh Bui	Project Coordinator & Report	Project coordination, time management, note-taking, Jira refreshing, final documentation, and report assignment tasks.
Aamna Khan, Magdalena Z.	Core Builder	Database schema design, unit tests (unittest), test coverage, hashing, and OOP
Amegna M., Hayley S.	Core Functionality	Flask routing, Console UI, CRUD operations for mood logs and journal entries, session management, and user authentication.
Ruth O.	Front-End, API & Specialist Topic,	Front-End, API integration, and hashing.
Ayesha G	Report & Presentation	Assist in testing UI, final testing, and final presentation

4.2 Agile Development Process

The team followed a Scrum-inspired process across four two-week sprints, using Jira as a Kanban board. Sprint ceremonies included planning (backlog estimation and prioritisation), bi-daily async stand-ups (Slack updates on progress and blockers), sprint review (feature demos against acceptance criteria), and retrospectives (process improvements and action points).

4.3. Git Workflow & Branch Strategy

Branch	Purpose
main	Protected production branch. Only merged into via approved Pull Requests. Always stable and deployable.
view-edit-delete-integration	Integration branch. Features are merged here first for team testing before being promoted to main.
Database	Database schema design and operations
API	API integration
Console-UI	Mood log CRUD routes and database operations User login, Login Menu, Exploratory console-based interface
Hashing	Hashing and app route paths
OOP	Implement OOP classes
khanh/cleanup	To clean up unnecessary branches

Pull Request Process

- All code was submitted via Pull Requests from feature branches into the main branch.
- A minimum of one peer review was required before merging — comments were left inline on GitHub.
- Merge conflicts were resolved collaboratively in pair sessions over Slack messages.

4.4 Tools & Libraries

Tool / Library	Purpose
Python 3.10+	Core programming language
Flask 3.x	Web framework — routing, Jinja2 templating, session management
MySQL	Relational database and Python connector for persistent storage
python-dotenv	Loads .env environment variables (DB credentials, Flask secret key)
requests	HTTP client library for ZenQuotes API calls
Werkzeug	Secure password hashing
unittest	Python standard library unit testing framework
Git & GitHub	Version control, branching, Pull Requests, code review
Jira	Kanban-style Agile board for sprint and backlog management
Postman	Manual API testing of Flask routes during development

5. Testing Strategy & Evaluation

5.1 Automated Testing

Once core functionalities were added, the test was established, and it focuses on the behaviour of the `db_utils.py` file. The test uses a mocked database without having a live data connection and returns expected status codes, response messages, and data structures. It includes: database connection, user creation, journal entries operation, etc.

5.2 Manual Testing

Manual testing was performed on the console UI due to automation limitations. Key scenarios tested:

- User registration and login (valid credentials, duplicate emails, invalid details)
- Journal CRUD operations (add, view, edit, delete)
- Searching and viewing entries
- Mood summary viewing
- Clear console error messages (no None returns)

5.3 Known Limitations

Due to time constraints, we did not create separate test files for `app.py` and `main.py`. Instead, testing was distributed across the codebase: Flask route behaviour is covered through integration tests, database behaviour through `db_utils.py` tests, and console UI behaviour through manual testing. While this approach does not achieve complete automated test coverage, it ensures the main user workflows and critical database and routing behaviours are validated.

5.4 Potential Testing Improvements

- Adding a dedicated `test_main.py` file for console input/output behaviour
- Adding a dedicated `test_app.py` file for more detailed Flask route testing
- Expanding integration tests for more edge cases
- Adding more tests after the OOP changes in `app.py`
- Adding coverage reporting with `pytest-cov`
- Adding automated tests for weather API failure handling
- Adding tests for database casing consistency across local environments

6. Conclusion

6.1 Final Results and Objectives

- Application Meets Proposal Goals
 - Flask + MySQL with secure authentication (password hashing)
 - Journal CRUD (create, view, edit, delete) fully functional
 - Search by keyword, filter by mood, sort by date
 - OpenWeather API integrated; weather auto-captured
 - Rule-based recommendations (mood, energy, free time, weather)

- Mood analytics (most common mood, frequency, patterns)
 - Console UI with intuitive menu navigation
- User Stories Addressed
 - Register/login securely & Logout
 - Create entries (mood, energy, free time, weather auto-captured), edit/delete
 - View, search, filter, sort entries
 - Receive recommendations
 - View mood analytics
- Runs Reliably Without Critical Errors
 - 2 unit tests (authentication, CRUD)
 - Comprehensive manual UAT all workflows
 - Full user journey tested end-to-end
 - OpenWeather API handles safely
 - Database queries reliable
 - No critical bugs; all features stable
- Summary: 4 weeks on schedule | 28 merged PRs | 100% task completion | 8 team members | Demo successful

6.2 Outcomes Against Original Aims

KindMind succeeds by solving real user problems: mood tracking + weather context + personalized recommendations help users understand emotional patterns. Users trust the system through secure, reliable design (zero critical errors); analytics drive engagement through visible progress. Business strengths: modular architecture scales easily, open APIs minimize costs, 100% on-time delivery, clean team execution. App establishes product-market fit in growing mental wellness space.

Time Constraints & Future Potential: Project completed in 4 weeks on tight deadline, prioritizing backend logic, core functionality and heavy testing over frontend polish. With extended timeline, team would enhance UI/UX with improved styling, interactive dashboards, and refined console experience, to elevate user engagement and app smoothness. Current foundation is solid and scalable; additional development time would directly translate to better user experience and market competitiveness.

6.3 Application Value

KindMind demonstrates that a production-quality mental health tool can be built by a small team using clean, well-structured Flask architecture. Key value propositions: accessibility (browser-based, free, no installation), privacy-first (no ads, encrypted passwords), developer-friendly (modular codebase, documented, industry-standard Git workflow), and extensible design for future features (analytics, therapist dashboards, mobile API).